Never Start from Scratch: Expediting On-Device LLM Personalization via Explainable Model Selection

Authors： Haoming Wang,Boyuan Yang, Xiangyu Yin, Wei Gao

University of Pittsburgh

Mobisys’25

Reporter: Hangshuai He

Intelligent Systems Lab @ PITT

Haoming Wang

PhD Student

University of Pittsburgh

Interests

- Efficient Geneartive AI
- Spatial Reasoning
- Explainable AI
- Federated Learning

Recent Papers:

- **[Mobicom' 25]** When device delays meet data heterogeneity in federated aiot applications
- **[AAAI' 25]** Tackling intertwined data and device heterogeneities in federated learning with unlimited staleness
- **[Mobisys' 25]** Never Start from Scratch: Expediting On-Device LLM Personalization via Explainable Model Selection

Wei Gao

Associate Professor

University of Pittsburgh

Interests

- On-device AI
- Mobile and embedded computing systems
- Mobile and connected health
- Cyber-physical systems and Internet of Things

Outline

- ***Background***
- ***System Design***
- ***Evaluation***
- ***Conclusion***

Outline

- ***Background***
- ***System Design***
- ***Evaluation***
- ***Conclusion***

Background

Why personalization?

- 手机上的LLM已经是常见配置
- 每个用户对询问内容的偏好、想要看到的回答风格以及日常使用时的上下文内容有很大不同
- 如果能个性化定制一个 personalization LLM（pLLM），会更符合用户需求

Background

Why personalization at edge?

- 最直接的做法：上传用户数据到云端做微调
- 这在工程上很方便，因为服务器算力强

- 但这样会带来严重的隐私问题
- 用户的聊天记录、写作习惯、个人偏好、上下文信息，都可能是敏感数据

- 因此更合理的路线是：在用户自己的手机本地完成微调
- 这样用户数据不需要离开设备

Background

Personalization at edge is hard

- 算力约束：即便是比较小的模型，如 Llama3.2-1B、Qwen2-0.5B，在手机上做微调也不便宜；以 Pixel 9 Pro 上的 Qwen2-0.5B 为例，即便使用 LoRA，一步训练也至少要数百毫秒
- 数据约束：有效微调通常需要不少样本，为避免 bias 和 overfitting，往往至少需要数千条训练样本；但普通手机用户每天产生的文本量有限，积累这么多本地数据很慢
- 一边是训练慢，一边是数据少；这使得“每个用户都从 base LLM 开始重新训”变得很不现实

Background

Motivation

- 理想化的场景是：每当一个用户在本地完成 personalization 后，他把得到的 personalized LLM（pLLM）上传到受信任云端缓存。之后新用户可以先从这些历史 pLLM 中找一个最接近自己的，再继续本地微调

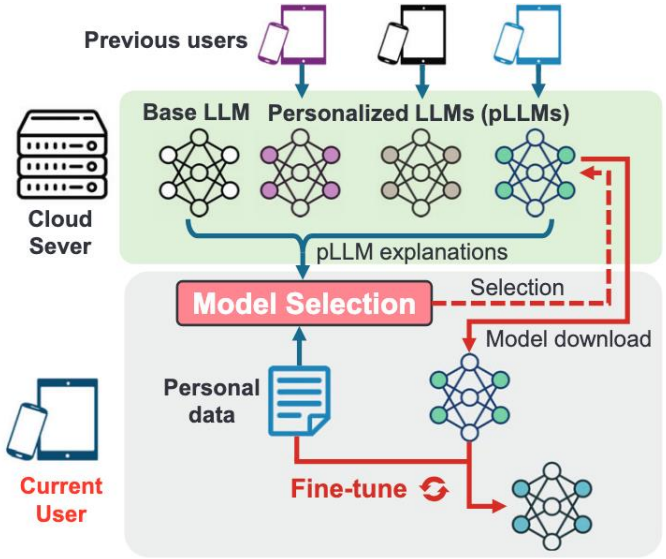
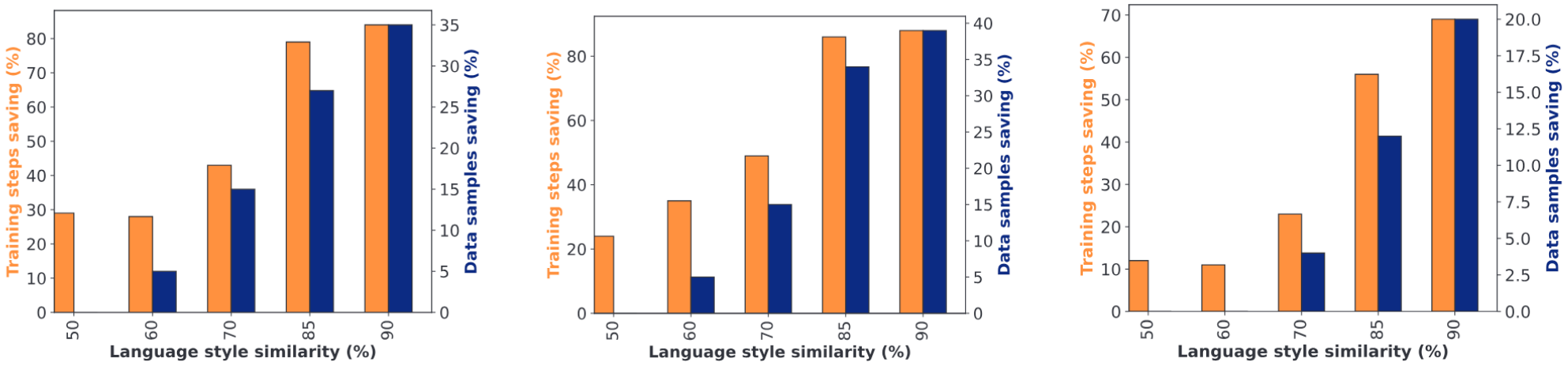


Figure 1: On-device LLM personalization via explainable model selection

Background

Motivation

- 从这些历史 pLLM 中找一个最接近自己的，再继续本地微调。



(a) Llama3.2-1B [24] (b) Qwen2-0.5B [98] (c) SmolLM-360M [10]

Figure 3: Model selection reduces the computational cost and the amount of data required for LLM fine-tuning

Background

Keypoint：历史 pLLM 可以复用，但怎么选对?

- 复用的难点在于，如何在不泄露隐私的前提下选到最合适的那个
- 最合适的 pLLM，应该是最接近当前用户需求的模型
- 但当前用户的个人数据不能上传到云端做匹配，因为这会违背 on-device personalization 的隐私前提
- 把所有 pLLM 都下载到手机逐个试也不现实，因为这会带来很大的通信开销和本地搜索计算开销，尤其当缓存的 pLLM 很多时更不现实

Problem：如何选到最合适的模型

Background

SOA工作

- 直接评估式模型选择
- 下载全部候选 pLLM，在本地用私人数据一个个跑推理/评估，再选最好的；或者用 Bayesian Optimization等优化算法来减少搜索

- 非可解释的特征编码匹配
- 用预训练文本编码器把 pLLM 输出与用户 personal data 编码成向量，再按相似度选模型。这类方法在计算负担上比较轻量，但缺乏明显的可解释性

- 可解释但优化负担比较大的分布比较
- 例如 prototype learning 。分别从 base LLM 与 pLLM 的输出分布中抽取底层 pattern，再比较差异。这类方法有一定解释性，但需要迭代优化，开销高

- 直接自然语言总结差异
- 用另一个 summarization LLM 比较 base/pLLM 的输出内容的样本对，并直接总结为自然语言解释。这类方法有自然语言解释，但作者认为这类总结可能有偏，而且难以量化，所以不适合做可靠的模型选择

Outline

- *Background*
- ***System Design***
- *Evaluation*
- *Conclusion*

System Design

Overview： XPerT

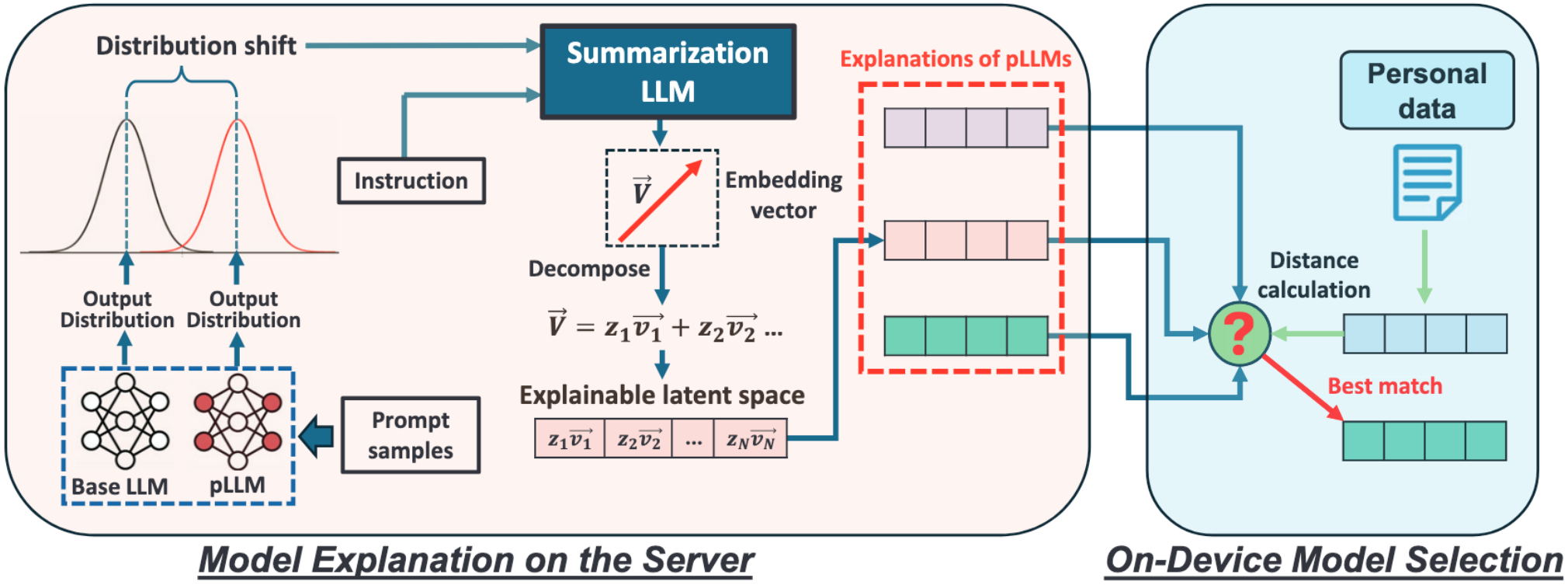


Figure 5: Overview of XPerT design

System Design

Overview： XPerT

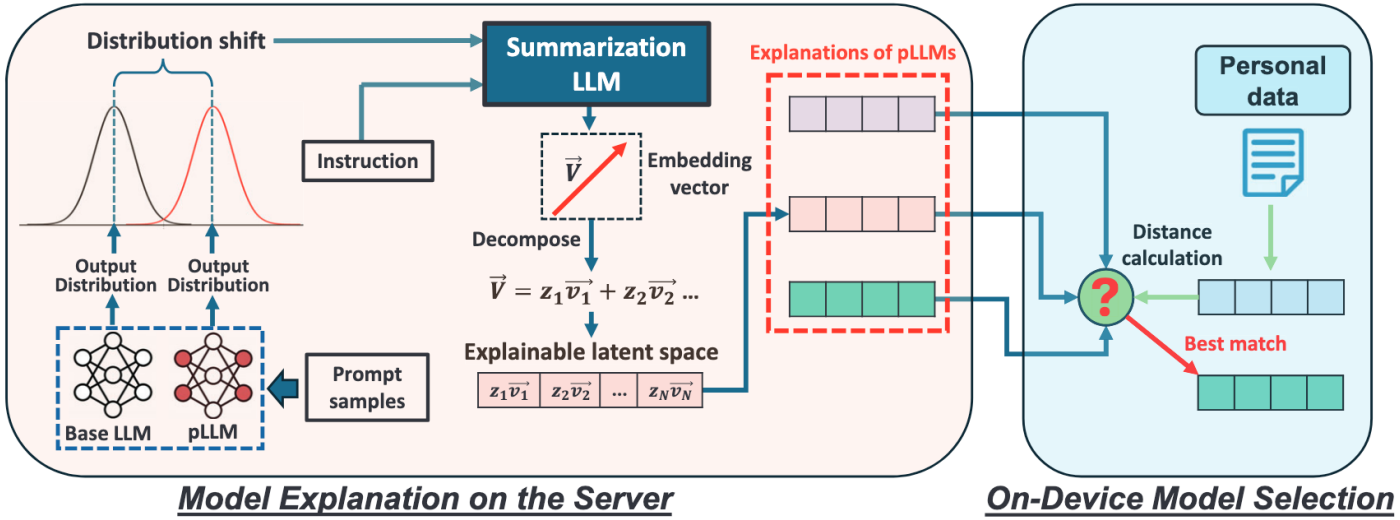


Figure 5: Overview of XPerT design

服务器端如何构造 pLLM 的 explanation

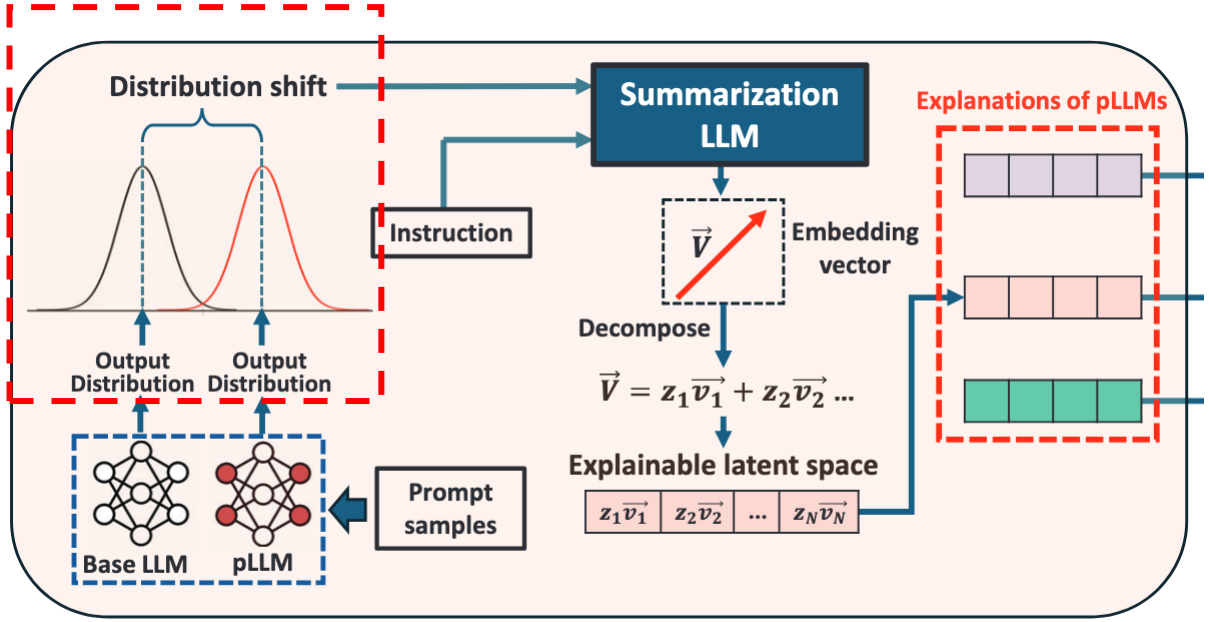
这个 explanation 为什么要进一步变成 explainable latent space

设备端如何用同样的表示去完成最终的 model selection

System Design

Model Explanation on the Server

目标：以一种可量化的方式，用自然语言描述pLLM与baseLLM输出分布之间的差异



Model Explanation on the Server

Given the two responses to the same question listed below, which language style is more characteristic of Response 1 than Response 2? Reply with only one adjective.

Question:
<QUESTION>
Response 1:
<**pLLM** RESPONSE>
Response 2:
<**baseLLM** RESPONSE>

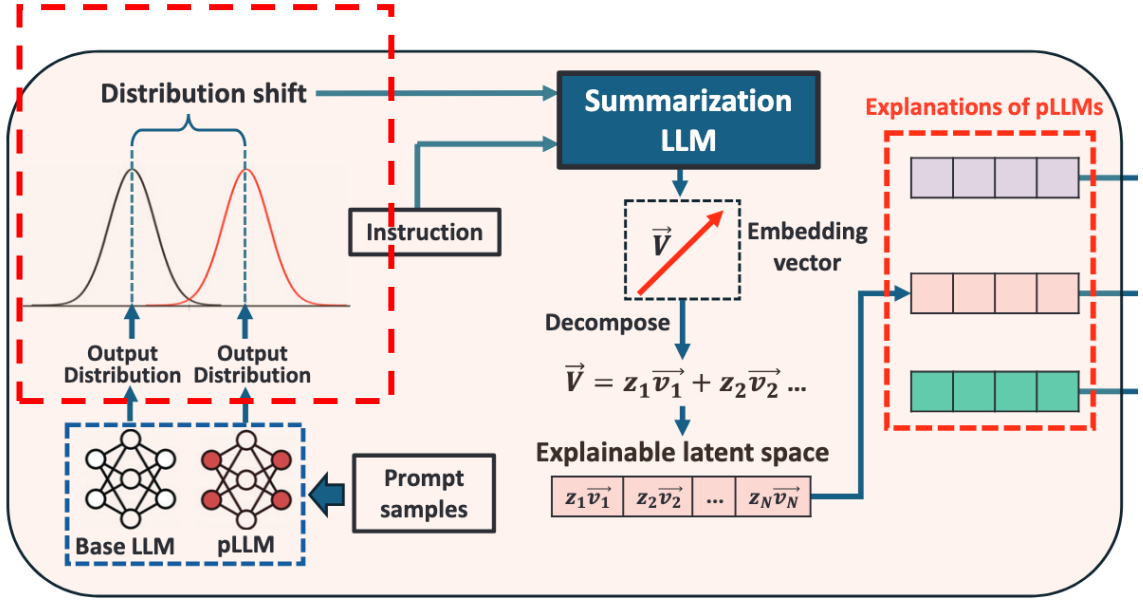
Figure 6: The instruction for the summarization LLM

formal **academic** **humorous**

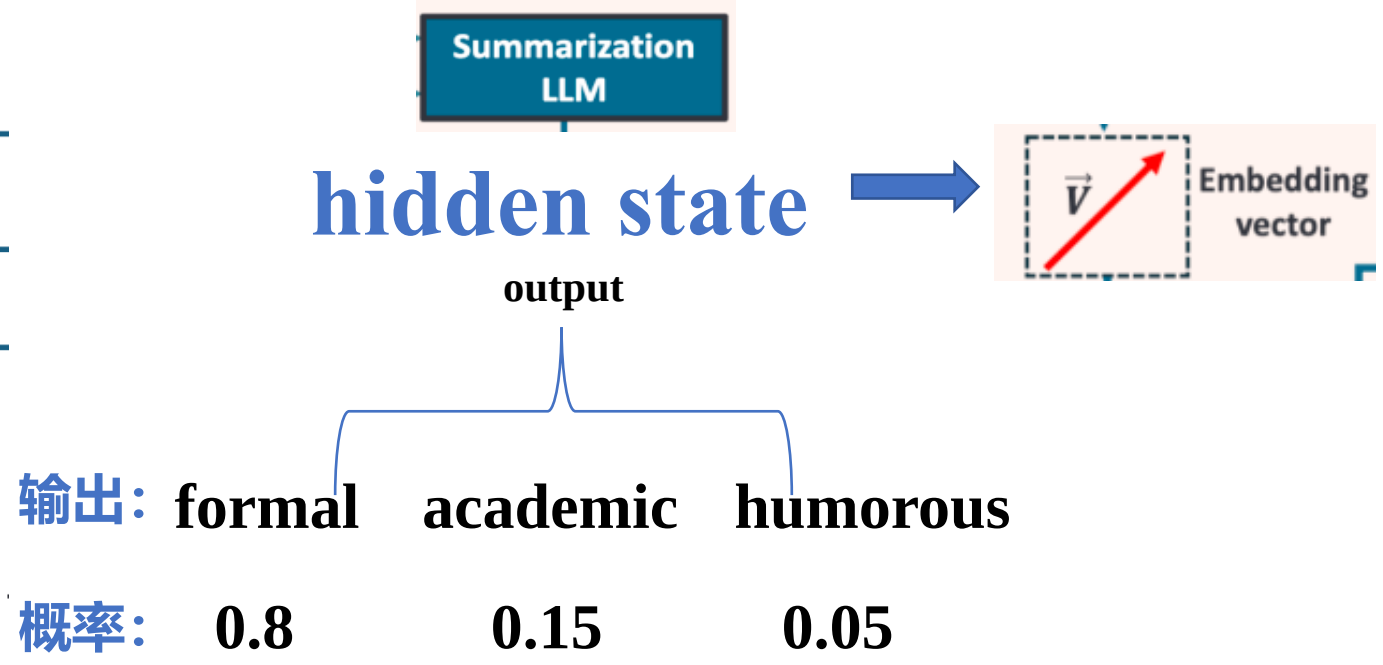
System Design

Model Explanation on the Server

目标：以一种可量化的方式，用自然语言描述pLLM与baseLLM输出分布之间的差异



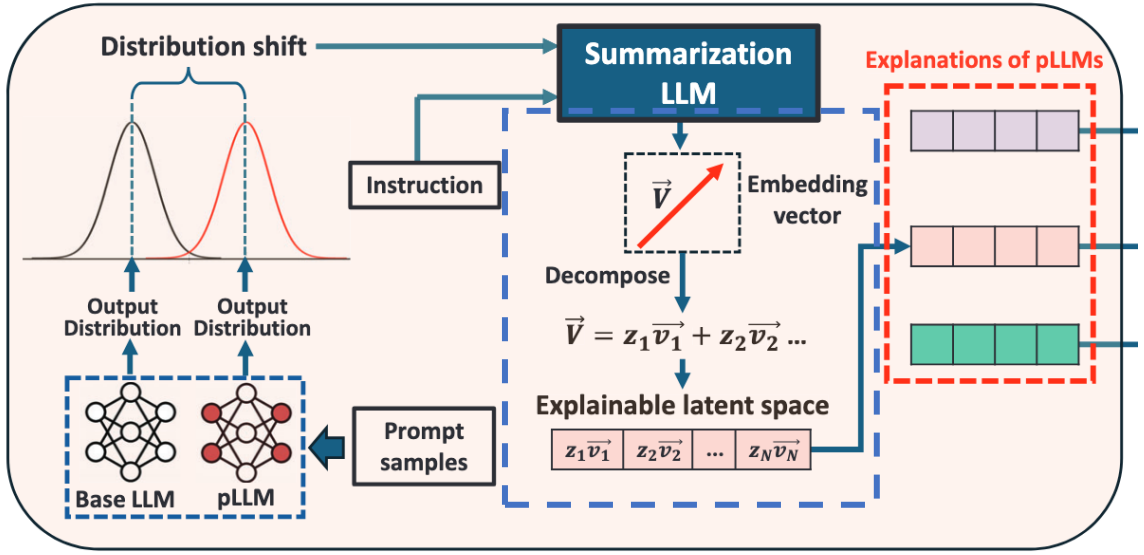
Model Explanation on the Server



前提：要求 Summarization LLM 只用一个形容词来描述输出差异

System Design

Model Explanation on the Server



Model Explanation on the Server

embedding vector V 依然是个黑箱，需要进一步分解

分解成一组正交子向量的线性组合：

词表中：	**formal**	**academic**	**humorous**
概率：	**0.8**	**0.15**	**0.05**

高概率词 → 候选风格方向 → 构造子向量

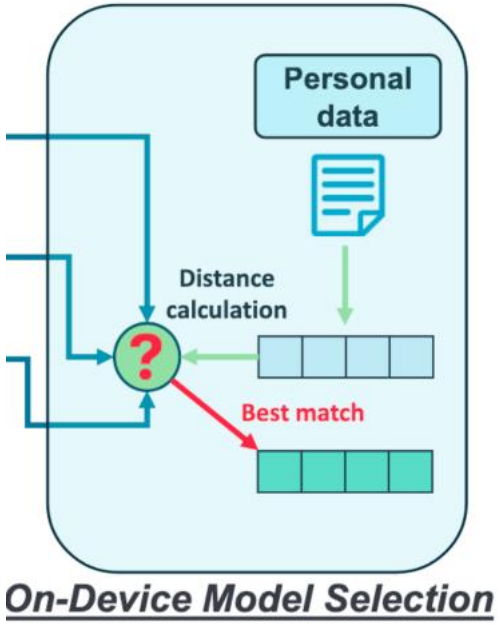
$$V \approx z1\ V_{formal} + z2\ V_{academic} + z3\ V_{humor}$$

我们也许可以用词表中的高概率词去构造子向量，从而用来表达pLLM的风格

System Design

On-Device Model Selection

$$\mathbf{pLLM1 = 0.5\ V_{formal} + 0.35\ V_{academic} + 0.15\ V_{humor}}$$



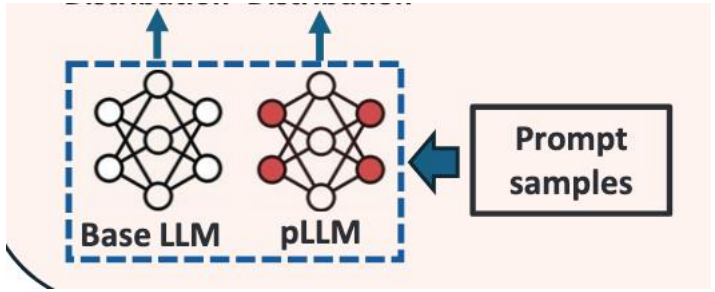
每个 pLLM 都能在这个 explainable latent space 里对应一个坐标

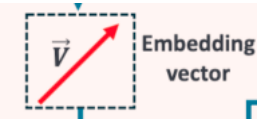
设备端：
把当前用户的 personal data 也映射到同一个空间里，然后比较距离，选出最近的那个 pLLM

比较很多完整模型谁更适合用户 → 在同一个可解释空间里，比较几个坐标点谁离用户最近

System Design

Model Explanation with Quantitative Measures





Embedding Vector 的提取

1. 对于服务器上每一个 pLLM，使用同一组 prompt 样本探测预训练的 base LLM 和该 pLLM
2. 读取 summarization LLM 在输出前的 hidden state，作为这个 pLLM 的 embedding vector
3. 问题：这个向量会不会受 prompt 数量和 prompt 分布影响？

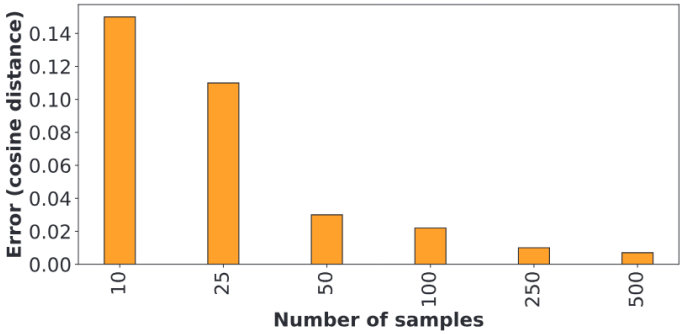


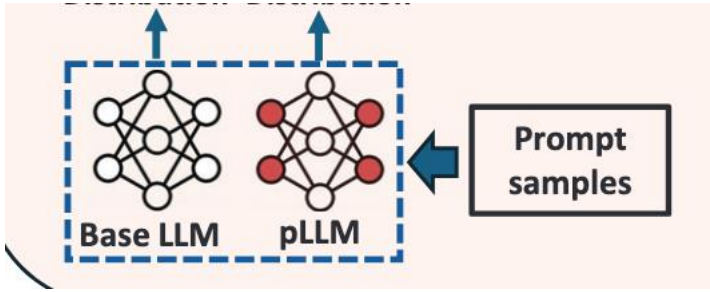
Figure 7: The error of embedding vector with prompt numbers, measured by cosine distance

embedding vector 是否正确，取决于这些 prompt 样本的数量和内容

1. 实验：考察 prompt 数量与 embedding vector 提取误差之间的关系
2. error定义：当前数量的 prompt 所提取出的 embedding vector，与使用足够大数量样本（如：1000 ）提取出的 embedding vector 之间的 cosine distance
3. 结论：prompt 到 50 个左右，结果已经很接近“大样本”提取出来的向量，所以 50 个 prompt 就够用了

System Design

Model Explanation with Quantitative Measures



Embedding Vector 的提取

1. 对于服务器上每一个 pLLM，使用同一组 prompt 样本探测预训练的 base LLM 和该 pLLM
2. 利用回答从 summarization LLM 的 embedding space 中提取 embedding vector
3. 问题：这个向量会不会受 prompt 数量和 prompt 分布影响？

pLLM model	Llama3.2 -1B	Qwen2 -0.5B	SmolLM -360M
Alpaca vs. LegalBench	0.042	0.054	0.060
LegalBench vs. SciQ	0.063	0.033	0.035
SciQ vs. Alpaca	0.029	0.041	0.032

Table 1: Differences of embedding vectors with prompt samples from datasets of different domains

Prompt 分布偏差实验

1. 问题：如果 prompt 的领域不同，比如法律问题和科学问题，会不会把 explanation 带来不同的结果
2. 实验：不同领域 prompt：Alpaca / LegalBench / SciQ，比较同一个 pLLM 在不同 领域prompt下提取出的 embedding vectors
3. 结果：cosine distance 很小
4. 结论：因为他们不是直接看 pLLM 的绝对输出，而是在看 pLLM 相对 base LLM 的差异，所以 prompt 本身造成的一部分共性偏差会被抵消

System Design

Decomposing the Embedding Vector

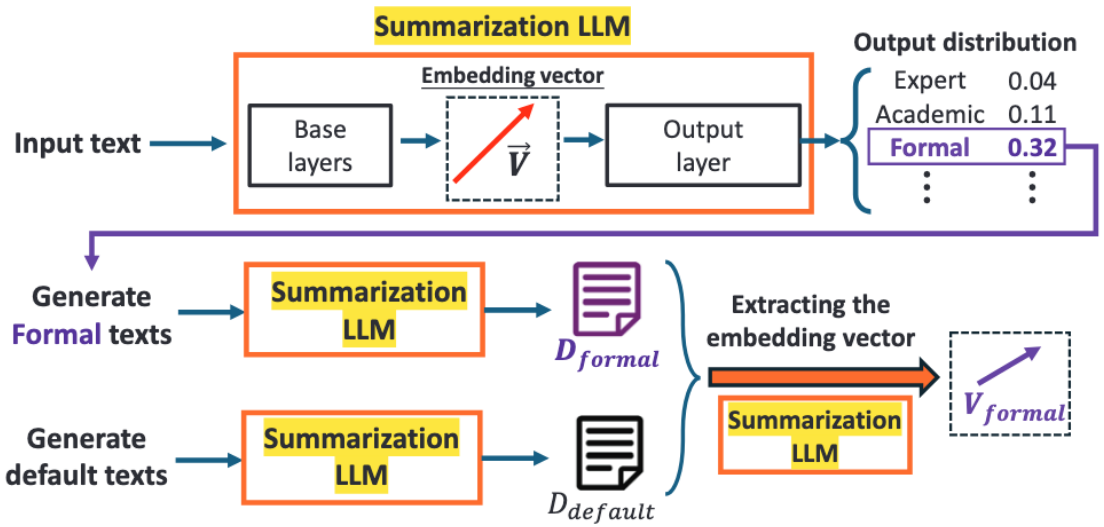


Figure 8: Decomposing the pLLM's embedding vector into a linear combination of orthogonal sub-vectors

词表上的输出分布

高概率的词，就选为candidate words，有用来描述pllm风格的可能

问题：怎么把一个 candidate word 变成一个 sub-vector ?

方法：

- 在同一组prompt上，让summarization LLM分别用默认风格/ candidate word 风格，来给出一组输出
- 复用之前提到的差异提取过程，对两组输出的整体风格偏移做 summary，提取一个总的 embedding vector
- 这个vector就被归纳到embedding space 里的一个方向
- 这些向量都来自同一个 summarization LLM 的内部表示

System Design

Decomposing the Embedding Vector

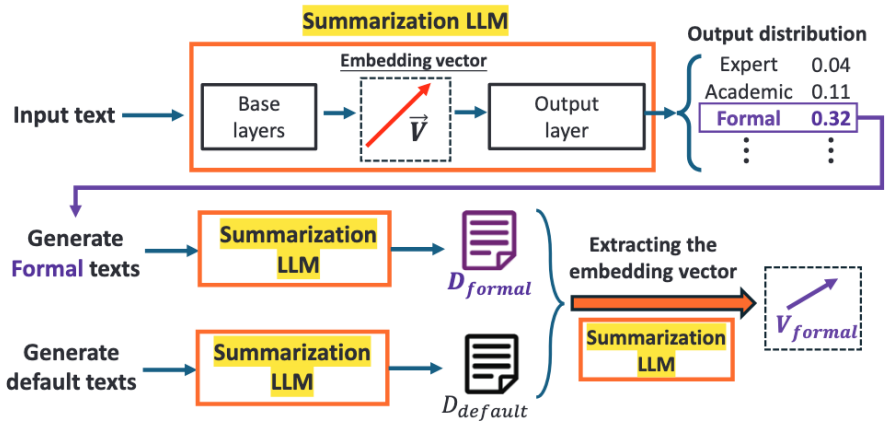


Figure 8: Decomposing the pLLM's embedding vector into a linear combination of orthogonal sub-vectors

Number of pLLMs	4	8	16
Number of orthogonal sub-vectors	12	19	37
Number of candidate words examined	92	145	280

Table 2: Numbers of orthogonal sub-vectors and candidate words needed for the decomposition

- 得到不同词的向量以后，就得到了一个可以用来表达每个pLLM的向量的坐标系
- $$z_i = \frac{V \cdot v_i}{\|v_i\|}$$
- 得到每个pLLM在某个方向上的强度

$$\mathbf{pLLM1 = 0.5\ V_{formal} + 0.35\ V_{academic} + 0.15\ V_{humor}}$$

- 所有缓存的 pLLM 都需要由这一套坐标系来表示

- 用当前已经有的那组 sub-vectors 去分解当前这个 pLLM 的向量
- 如果分解后的重建误差太大，那就说明这组坐标系还不够
- 继续从词表中添加新的 candidate word，检查是否与现有坐标系词汇正交，如果正交，就加入坐标系，继续尝试分解

System Design

Computationally Efficient Sub-vector Generation

Number of pLLMs	4	8	16
Number of orthogonal sub-vectors	12	19	37
Number of candidate words examined	92	145	280

Table 2: Numbers of orthogonal sub-vectors and candidate words needed for the decomposition

- 观察：如果有16个pLLM，就需要测试280个 candidate word，计算开销成本很大
- 假设：可以让Summarization LLM直接给出 candidate word 对应的sub vector
- 问题：怎么保证直接生成仍能准确？

# of training samples	50	100	200	300	400
10 trainable tokens	0.25	0.12	0.07	0.07	0.08
25 trainable tokens	0.19	0.09	0.06	0.05	0.06
50 trainable tokens	0.29	0.15	0.07	0.07	0.05

Table 3: The error of computing the sub-vector with prompt-tuned summarization LLM

- Design：offline prompt tuning
- 给定 prompt 形式：[prefix, candidate word, suffix]
- prefix / suffix 是 语义可以被训练的tokens
- 用刚才提到的，花费大量时间生成的 sub-vector 当 ground truth
- 得到： prompt-tuned summarization LLM ，在这样的prompt组合下，summarization LLM 看到candidate word 就给出接近的 sub-vector
- 结论：200 个训练样本基本足以使 prompt-tuned summarization LLM 对每个candidate word 给出接近的 sub-vector

System Design

On-Device Model Selection

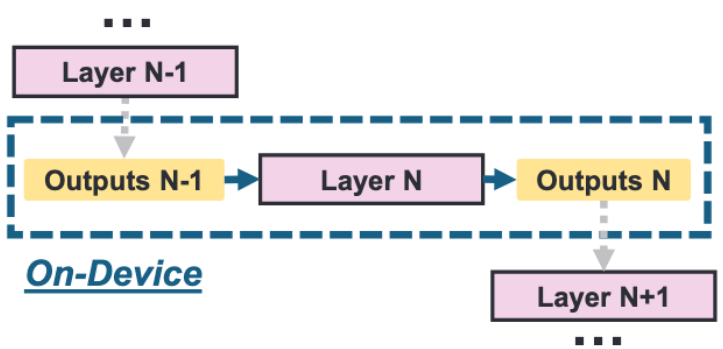


Figure 10: Fitting the summarization LLM to the user device's memory constraint

Device	Inference time	Loading & switching time
OnePlus 12R	10.1min	9.87s
Pixel 9 Pro	14.2min	12.24s

Table 4: Costs of on-device model loading and switching

- 观察：已知每个pLLM的坐标，下一步要得到用户数据的坐标。但用户本地数据需要在本地映射到 latent space，就必须运行 summarization LLM；但 summarization LLM 太大，手机内存装不下。
- 例子：Llama3.1-8B
  INT8 下仍然要 10–12GB 空闲内存
  大多数手机没有
- 做法：将 summarization LLM 进行分块，每次只对其中一层执行推理。这一层的输出会被缓存到内存中，并作为下一层的输入。
- 实验证明：使用该方式进行推理，内存降到 2.8GB，只剩原来的 23%。而层之间的加载与切换时间不到完整 LLM 推理时间的 2%，而且这种切换不会对最终生成的 sub-vectors 造成任何影响

System Design

On-Device Model Merging

- 观察：已知用户数据的坐标。但最近的单个 pLLM 不一定足够像用户
- 例子：假设我们现在只有两个主要风格维度：formal 和 humorous。用户坐标（0.5humorous，0.5 formal），pLLM A 比较正式，坐标可以理解成 (0.9，0.1）pLLM B 比较幽默，坐标可以理解成 (0.2，0.8）。任何一个pLLM，与用户坐标的余弦距离都不够近
- 做法：通过数学计算，发现如果两个 pLLM 的加权组合，比任何单独一个 pLLM 都更接近用户坐标，比如：
- 0.3 × A + 0.7 × B，那就根据这个权重，在参数上组合两个模型。
- 例如：每个 pLLM 相比 base model，都有一个偏移量，合并模型时，就把两个偏移量一起加到 base model 上。合并权重（0.3,0.7) 就来自 latent space 中“两个 pLLM 该占多大比例”的估计。
- 解释：这是一个经验性近似，不是严格数学等价

Outline

- *Background*
- *System Design*
- ***Evaluation***
- *Conclusion*

Evaluation

Experimental Settings

Cloud server： 4 Nvidia H100 GPU

summarization LLM： Llama3.1-8B-Instruct

Base LLM/pLLM： Llama3.21B , Qwen2.5-0.5B ， SmolLM-360M

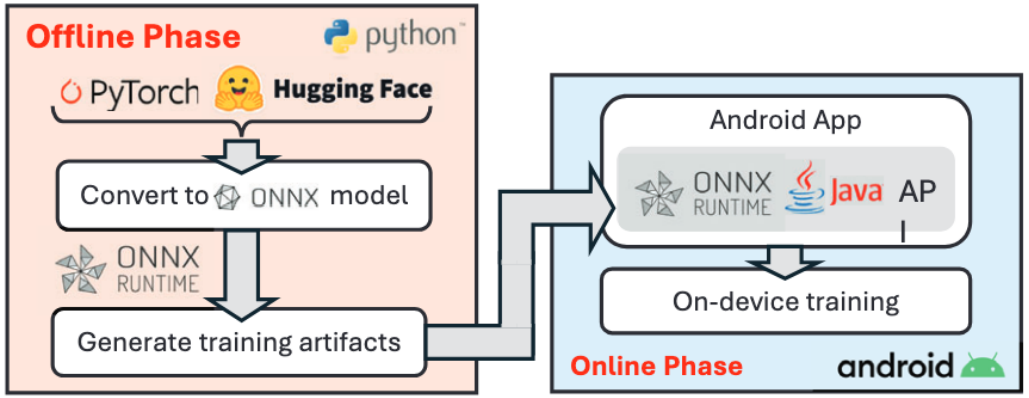


Figure 11: Fine-tuning LLM on smartphones

Evaluation

	Llama-3.2-1B on One Plus 12R				Qwen2-0.5B on Pixel 9 Pro				SmolLM-360M on Pixel 7			
Synthetic	Acc	FT-time	Energy	Data	Acc	FT-time	Energy	Data	Acc	FT-time	Energy	Data
From scratch	-	97.8min	15.7kJ	0%	-	26.4min	6.2kJ	0%	-	93.6min	18.0kJ	0%
30% similarity	25.0%	92.4min	14.9kJ	4.6%	28.6%	23.1min	5.5kJ	2.4%	17.8%	89.4min	17.1kJ	3.6%
50% similarity	53.6%	81.8min	13.3kJ	16.7%	64.2%	21.9min	5.2kJ	14.4%	57.1%	84.4min	16.7kJ	9.9%
70% similarity	85.7%	56.7min	9.0kJ	17.1%	82.1%	13.4min	3.1kJ	15.2%	82.1%	70.8min	13.6kJ	4.4%
80% similarity	96.4%	32.9min	5.3kJ	24.7%	92.9%	5.8min	1.4kJ	33.7%	89.3%	56.2min	10.9kJ	13.5%
90% similarity	96.4%	17.9min	2.8kJ	35.7%	100%	4.4min	1.0kJ	33.2%	89.3%	26.2min	5.1kJ	21.7%
Real-world	Acc	FT-time	Energy	Data	Acc	FT-time	Energy	Data	Acc	FT-time	Energy	Data
From scratch	-	141.6min	23.5kJ	0	-	41.2min	9.40kJ	0	-	149.4min	26.5kJ	0
30% similarity	16.4%	124.2min	20.6kJ	7.2%	16.9%	38.5min	9.02kJ	3.7%	8.3%	140.7min	25.8kJ	5.2%
50% similarity	60.7%	99.0min	16.6kJ	16.4%	48.3%	28.4min	6.58kJ	10.0%	40.7%	124.4min	24.1kJ	9.1%
70% similarity	88.2%	61.2min	10.1kJ	29.0%	80.5%	22.7min	4.34kJ	30.6%	76.3%	96.6min	17.2kJ	15.1%
80% similarity	88.9%	37.8min	6.27kJ	42.1%	84.7%	19.0min	19.0kJ	38.2%	88.1%	74.4min	12.9kJ	27.2%
90% similarity	96.7%	27.5min	5.29kJ	47.4%	92.4%	7.1min	1.61kJ	51.4%	92.8%	56.7min	10.7kJ	28.3%

Table 5: XPerT's performance of on-device LLM personalization with different levels of similarity between pLLMs and on-device personal data. Performance is measured by model selection accuracy ('Acc'), wall-clock time ('FT-time') and energy consumption ('Energy') of on-device fine-tuning, and the improvement of data efficiency ('Data')

Acc ： 系统对相似度级别对应的目标 pLLM的正确选取率

FT-time：端侧 fine-tuning 的花费时长

Energy ： 端侧 fine-tuning 的能耗

Data：端侧 fine-tuning 的Data efficiency

Evaluation

	Llama-3.2 1B on One Plus 12R			Qwen2-0.5B on Pixel 9 Pro			SmolLM-360M on Pixel 7		
Synthetic	BLEU	ROUGE-1	ROUGE-L	BLEU	ROUGE-1	ROUGE-L	BLEU	ROUGE-1	ROUGE-L
From scratch	0.13	0.32	0.23	0.14	0.33	0.25	0.09	0.35	0.23
30% similarity	0.13	0.33	0.21	0.15	0.33	0.21	0.09	0.32	0.22
70% similarity	0.12	0.33	0.21	0.13	0.32	0.24	0.10	0.34	0.21
90% similarity	0.15	0.33	0.22	0.13	0.32	0.25	0.11	0.35	0.23
Real-world	BLEU	ROUGE-1	ROUGE-L	BLEU	ROUGE-1	ROUGE-L	BLEU	ROUGE-1	ROUGE-L
From scratch	0.18	0.39	0.30	0.07	0.31	0.27	0.06	0.29	0.26
30% similarity	0.18	0.39	0.30	0.07	0.31	0.27	0.06	0.29	0.26
70% similarity	0.19	0.39	0.28	0.07	0.31	0.25	0.06	0.28	0.27
90% similarity	0.20	0.38	0.29	0.07	0.31	0.28	0.06	0.28	0.29

Table 6: XPerT's performance in text generation task, when using personalized LLM fine-tuned from the selected pLLM and from scratch, measured by the BLEU and ROUGE scores

比较两种起点方式，最后生成文本的质量是否有差别

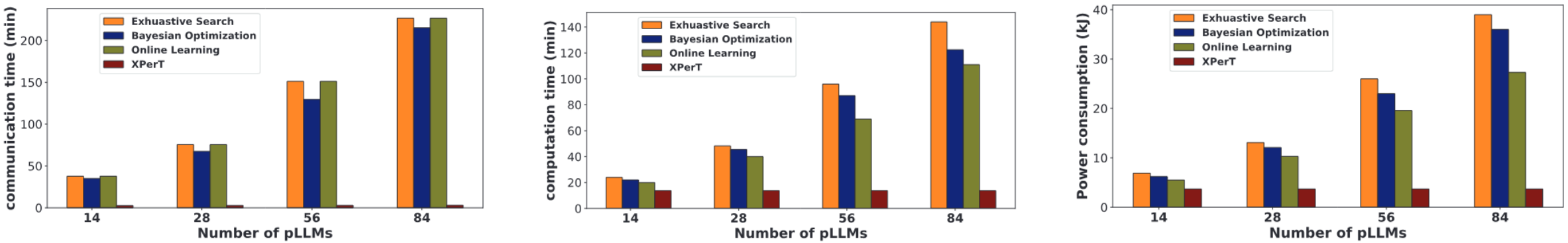
Evaluation

	Llama-3.2-1B on One Plus 12R			Qwen2-0.5B on Pixel 9 Pro			SmolLM-360M on Pixel 7		
Synthetic	t_{comm}	t_{comp}	Power	t_{comm}	t_{comp}	Power	t_{comm}	t_{comp}	Power
Exhaustive Search	1.26h	48.3min	13.1kJ	29.9min	19.6min	4.51kJ	23.8min	12.7min	1.54kJ
Bayesian Optimization	1.26h	45.5min	12.1kJ	27.8min	17.3min	4.00kJ	22.9min	12.1min	1.46kJ
HyperBand	1.26h	40.0min	10.3kJ	29.9min	16.2min	3.76kJ	23.8min	10.6min	1.29kJ
XPerT	2.7min	13.8min	3.7kJ	1.07min	10.8min	2.50kJ	0.85min	10.1min	1.22kJ
Real-world	t_{comm}	t_{comp}	Power	t_{comm}	t_{comp}	Power	t_{comm}	t_{comp}	Power
Exhaustive Search	1.12h	42.8min	11.4kJ	26.7min	16.2min	3.75kJ	21.2min	11.3min	1.54 kJ
Bayesian Optimization	1.03h	39.3min	10.7kJ	24.6min	15.6min	3.75kJ	20.4min	10.7min	1.37kJ
HyperBand	1.12h	35.9min	9.6kJ	26.7min	14.4min	3.25kJ	21.2min	10.3min	1.12kJ
XPerT	2.7min	13.8min	3.7kJ	1.07min	10.8min	2.50kJ	0.85min	10.1min	1.22kJ

Table 7: Communication cost (t_{comm}), computation cost (t_{comp}) and power consumption of on-device model selection

为了选出那个最合适的 pLLM，设备端本身要花多少代价

Evaluation



(a) Communication Cost (b) Computing Cost (c) Power Consumption

Figure 12: Scalability of XPerT's on-device model selection with different numbers of cached pLLMs on the server

当服务器上缓存的历史 pLLM 数量越来越多时，XPerT 的端侧 model selection 开销变化

Evaluation

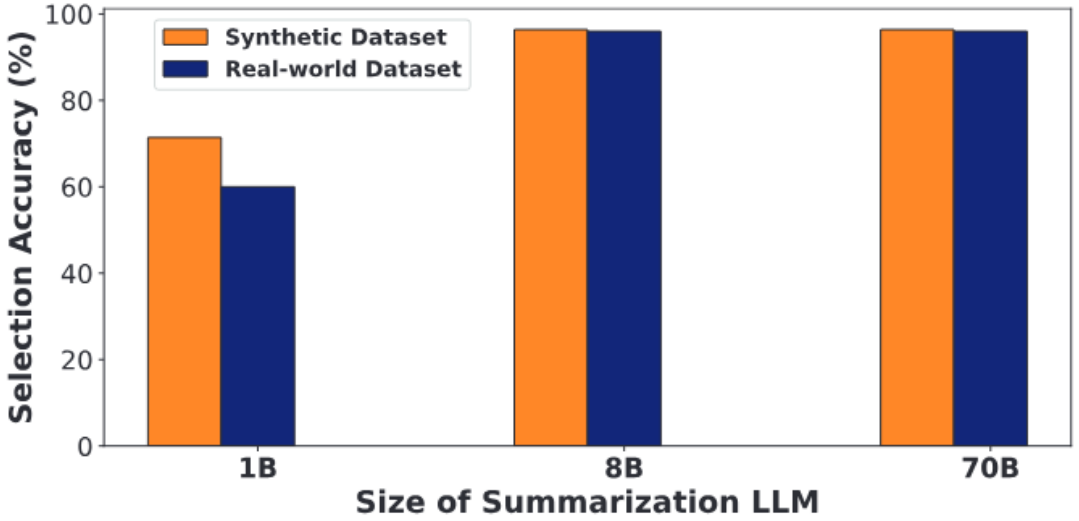


Figure 13: Accuracy of model selection with variants of Llama3.1/3.2 models as summarization LLM

把不同大小的 Llama 模型拿来做 summarization LLM，最终 model selection accuracy

Evaluation

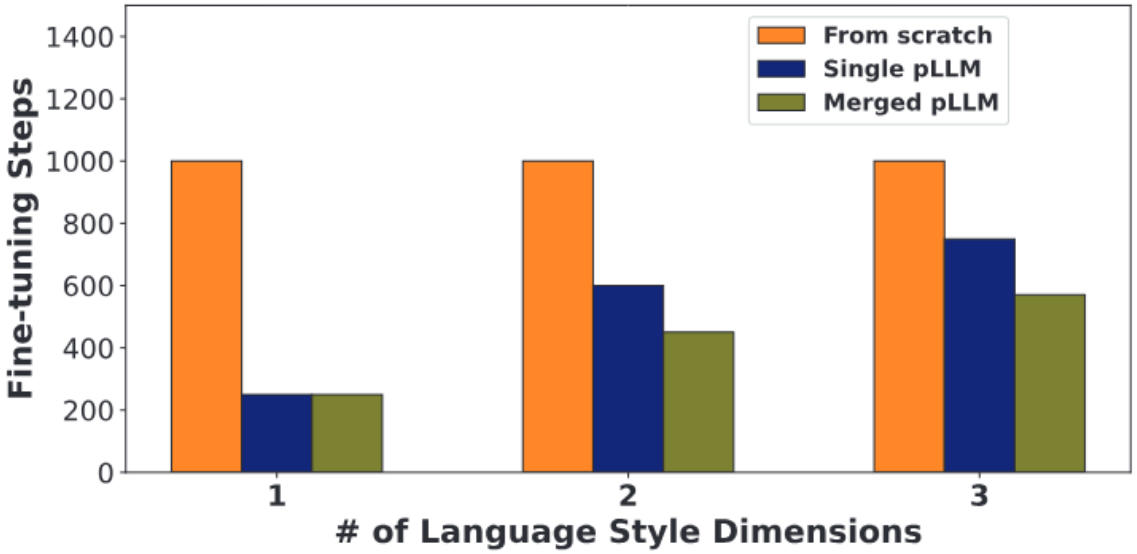


Figure 16: Effectiveness of on-device model merging

验证端侧模型合并model merging到底有没有必要、有没有效果

Evaluation

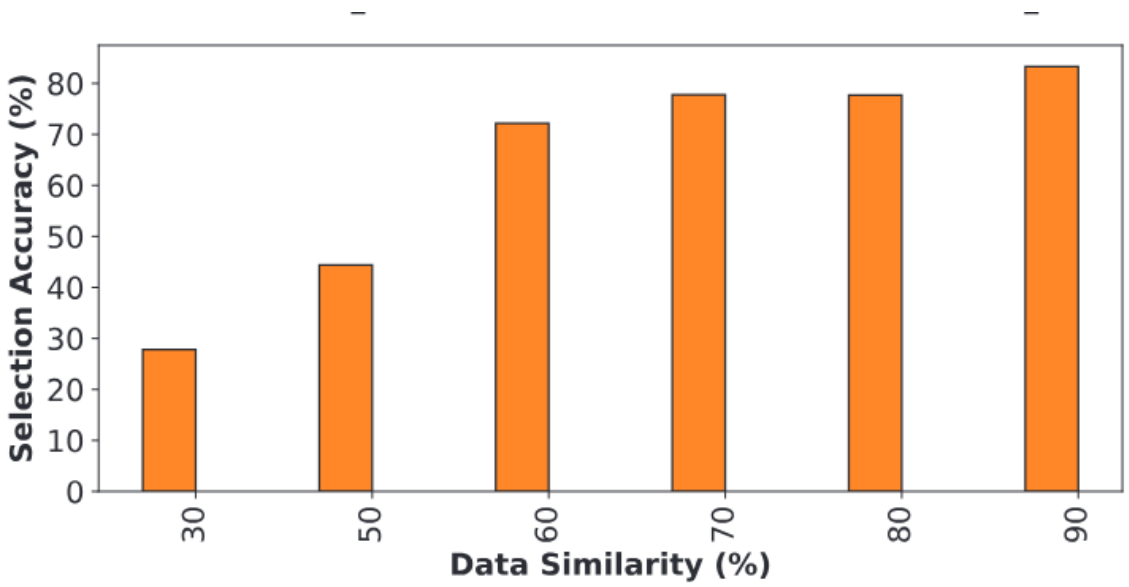


Figure 18: Accuracy of pLLM selection on the image generation task

验证 XPerT 不只适用于文本个性化，也能迁移到图像生成这种别的模态。

Outline

- *Background*
- *System Design*
- *Evaluation*
- ***Conclusion***

Conclusion

➢ 这个paper有什么问题，基于这个paper还能做什么？

- 基于一个硬假设，服务器上已经有多样化、高质量的很多历史 pLLM：
  - 其实和我的个人项目类似，这篇论文的收益，本质上建立在有一个足够丰富的历史 pLLM 库之上。
  - 所以它这个系统，其实是有前提的，需要先有 足够多的pllm积累，之后 XPerT 才能很好工作。
  - 也就是说，从 0 到 1 的过程，实际上没有被很好地解决。

- 默认用户端数据足以形成一个稳定、完备的风格表示：
  - 作者提到，最起码需要上千条“短信”数据，这个其实比较难积累。
  - 如果本地数据本身很少，XPerT 估计出来的用户风格坐标也可能不稳定。
  - 而且，用户数据在不同场景，本身也可能是多上下文、多场景、时间变化的混合分布。

Conclusion

- 这个paper提到的idea，能不能用在自己的方向/project上面?
  - 文章用到的这个构造路由空间，然后做的可解释方法，其实和我们的预期很像。
  - 并且我们的方法，在这篇论文的角度看来，似乎还欠缺可解释性。
  - 我认为我们的【手动设计emb，然后自己做cosine匹配的方法】，也许可以优化为作者文章的设计。
- 这个paper能不能泛化?
  - 这篇论文提出的，其实是一种范式。基于一个base和由此衍生出来的内容，主要存储这些衍生内容相对base的差异。而为了达到**可解释性**这样一个热点目标，可以由LLM等工具，将其解构到可解释空间中，从而减小规模/开销，加速计算。
  - 如果放到搜索式的张量优化范式当中，我觉得（聚类+选出范围内最代表的硬件作为base范式，剩余的簇内硬件的评估模型仅保存一种数据推导的范式，做类似的方式，也许可以引导（搜索式的评估模型）在不同硬件间的泛化。

Never Start from Scratch: Expediting On-Device LLM Personalization via Explainable Model Selection

Authors： Haoming Wang,Boyuan Yang, Xiangyu Yin, Wei Gao

University of Pittsburgh

Mobisys’25

Reporter: Hangshuai He